CASE STUDY

Fuds Field Service Management Dashboard

Type: Built & Shipped

Domain: AI Automation · Operations

Status: Live on AWS Amplify

Built & Shipped | AI Automation · Operations | Product Portfolio | Fuds

Type: Built & Shipped | **Domain:** AI Automation · Operations | **Status:** 🟢 Live on AWS Amplify

🔗 View Live Dashboard

TL;DR

> *After launching the Fuds website and generating 70 leads/month, the operations side was still manual and chaotic — 30+ minutes of coordination per order. I built and deployed an AI-automated field service dashboard solo, zero budget, using vibe coding. Shopify webhook → admin panel → instant WhatsApp to customer and technician. Coordination time: near zero.*

The Problem

```
BEFORE: What happened when a service request came in

Customer fills form → someone reads the email
→ manually finds available technician
→ calls technician to confirm
→ calls customer back to confirm time
→ hopes both people show up

Time per order: 30+ minutes
Failure modes: missed calls, wrong assignments, no customer confirmation
```

With leads growing from 20 to 70/month after the website launch, this manual process was becoming a critical bottleneck. More leads = more coordination overhead = slower response times = lower trust at the most sensitive moment in the customer relationship.

The core problem:

An 8-person company with no CRM, no dispatch tool, and no budget for either was handling growing lead volume with phone calls and WhatsApp messages.

Why I Built It

This wasn't in the original brief. I identified the operations problem independently while tracking the post-inquiry journey.

The observation: customers who received a fast, professional response were significantly more likely to convert and return. Customers who waited hours for confirmation often went cold or called a competitor.

The constraint: zero engineering budget, one shared developer already occupied with Shopify maintenance, no CRM subscription available.

The decision: build it myself using vibe coding tools (Claude + Antigravity) — a production-grade system, not a prototype.

My Role

- Solo product owner + builder
- System design and workflow architecture
- Vibe coded the dashboard using Claude + Antigravity
- Shopify webhook integration setup
- WhatsApp Business API configuration
- AWS Amplify deployment
- Testing and iteration post-launch

System Architecture

```
AUTOMATED FLOW (post-build)

[Customer submits booking form on Shopify website]
        ↓
[Shopify webhook fires → Dashboard receives order in real-time]
        ↓
[Admin sees new request in dashboard panel]
        ↓
[Admin assigns technician with one tap]
        ↓
    ┌──────────────────────────────────┐
    ↓                                  ↓
[Customer receives WhatsApp]    [Technician receives WhatsApp]
- Technician name               - Customer name + address
- Aadhaar-verified credentials  - Service details
- Contact number                - Appointment time
- Estimated arrival time        - Customer contact number
```

Key Design Decisions

Decision 1: WhatsApp over email

Tier 2 and non-metro customers have near-100% WhatsApp usage. Email open rates for service confirmations are low. WhatsApp messages get read within minutes. This was a deliberate product decision informed by user behaviour observation.

Decision 2: Aadhaar-verified technician details in customer message

User research at Fuds and Shaadi.com had consistently shown trust as the #1 barrier in service contexts. Showing the technician's Aadhaar-verified name and credentials in the confirmation message directly addressed the "who is coming to my home?" anxiety.

Decision 3: Dual notification (customer AND technician simultaneously)

Previous process: customer confirmed first, then technician called. This created coordination lag. Simultaneous notification eliminated the middle step entirely.

Decision 4: AWS Amplify for deployment

Free tier, managed infrastructure, no devops overhead. Right tool for the constraint.

Tools Used

LAYER	TOOL
Vibe coding	Claude (Anthropic) + Antigravity
Webhook integration	Shopify Webhooks
Messaging	WhatsApp Business API
Hosting	AWS Amplify
Auth	AWS Cognito
Data layer	AppSync GraphQL

Impact

METRIC	BEFORE	AFTER
Order coordination time	~30 minutes/order	Near zero
Customer confirmation speed	Hours	Seconds
Technician briefing	Phone call required	Automatic
Human error in assignment	Frequent	Eliminated
Repeat customer retention	Unmeasured	~40%

Why 40% retention?

The automated confirmation system delivered a professional, instant, verified response at the exact moment of peak customer anxiety — immediately after submitting a home security installation request. The experience of receiving a verified technician's details within seconds created trust that translated into repeat business.

Constraints I Worked Within

- **Zero engineering budget** — built entirely with vibe coding tools, no hired developers
- **8-person team** — no dedicated operations staff, no CRM, no project management tool
- **Non-technical operators** — dashboard had to be usable by anyone on the team without training
- **Production requirements** — this wasn't a prototype; real customers and real technicians depended on it working correctly

Key Learnings

1. **Constraints produce better product decisions.** No budget forced a solution that was more elegant and maintainable than a full CRM implementation would have been.
2. **The confirmation moment is a product feature.** What happens in the 60 seconds after a customer submits a service request determines whether they trust you. Automating that moment with verified information was the highest-leverage intervention.
3. **Vibe coding is a legitimate PM superpower.** Building this solo — from system design to AWS deployment — with AI tools took weeks, not months. A PM who can ship production systems has a fundamentally different relationship with engineering trade-offs.

Case study authored by Hiiren Rathod · linkedin.com/in/hiirenrathod

---